

MANUAL TÉCNICO · V1.0 · UPC · INGENIERÍA DE SISTEMAS

CardioAlert Manual técnico

Cómo está construida la app y cómo se integraron el sensor Polar H10, la inteligencia artificial y la nube.



PROYECTO

Solución de soporte clínico
con wearables e IA

AUTORES

Pérez Pacora, Antonio · Sara
García, Darwin

FECHA

Septiembre de 2026

CONTENIDO

1. Visión general del sistema
2. Tecnologías utilizadas
3. Integración del sensor Polar H10
4. Del sensor al diagnóstico
5. Conexión con Supabase
6. Backend y API de inteligencia artificial
7. Configuración
8. Compilación del APK
9. Despliegue en la nube
10. Guía de uso por rol
11. Solución de problemas
12. Referencias

01 Visión general del sistema

CardioAlert es una aplicación móvil de soporte a la decisión clínica para equipos de emergencia. Una banda **Polar H10** colocada en el pecho del paciente mide su electrocardiograma (ECG) y lo transmite por **Bluetooth** al celular del paramédico. La app analiza la señal con un modelo de inteligencia artificial que la clasifica como **Normal**, **Fibrilación auricular** o **Isquemia miocárdica**, explica qué parte del ECG motivó la decisión y, si hay una anomalía, permite enviar una alerta que el médico del hospital recibe en tiempo real.

Componentes

COMPONENTE	QUÉ ES	DÓNDE CORRE
Sensor	Banda Polar H10 (ECG de una derivación)	Pecho del paciente
App móvil	Aplicación React Native con módulo nativo Android	Celulares Android 13 o superior
Backend	API REST en Node.js y Express	Railway
API de IA	Servicio en Python con FastAPI y TensorFlow	Railway
Base de datos	Supabase: PostgreSQL, autenticación y tiempo real	Supabase (región São Paulo)
Entrenamiento	Notebook de Google Colab con GPU	Google Colab (solo al entrenar)

Roles

ROL	QUÉ HACE EN LA APP
Paramédico	Conecta el Polar H10, monitorea al paciente, revisa el diagnóstico y la explicación de la IA, y envía alertas al hospital
Médico	Recibe las alertas en tiempo real en el Monitor Hospital y las acepta o deriva
Administrador	Crea las cuentas de paramédicos y médicos

Alcance

CardioAlert es una herramienta de **apoyo** a la decisión clínica: no reemplaza el criterio del profesional. El modelo se entrena y valida con bases de datos públicas de PhysioNet, no con pacientes del proyecto.

02 Tecnologías utilizadas

Cada pieza del sistema se construyó sobre tecnologías de uso estándar en la industria. Esta tabla indica qué se usó, en qué versión y para qué.

ÁREA	TECNOLOGÍA	VERSIÓN	USO EN CARDIOALERT
App móvil	React Native (proyecto sin Expo)	0.87.1	Interfaz y lógica de la app
Lenguaje de la app	TypeScript	6.0	Código de la app con tipos
Sensor	Polar BLE SDK (oficial de Polar)	8.3.0	Buscar, conectar y leer el ECG del Polar H10
Módulo nativo	Kotlin + Kotlin Coroutines	2.2 · 1.10.2	Puente entre el Polar SDK y React Native
Bluetooth	Bluetooth Low Energy (BLE)	—	Comunicación sensor ↔ celular
Gráficos	react-native-svg	15.15	Trazo del ECG y mapa de la IA
Ubicación	@react-native-community/geolocation	3.4	GPS de la ambulancia en las alertas
Base de datos y login	Supabase (@supabase/supabase-js)	2.x	Autenticación, datos y tiempo real
Backend	Node.js + Express + Zod	22 · 5.1 · 4.1	API de sesiones, alertas y usuarios
API de IA	Python + FastAPI + TensorFlow	3.12 · 0.141 · 2.20	Clasificación del ECG y explicación
Datos de entrenamiento	PhysioNet: MIT-BIH, AFDB, European ST-T	—	ECG anotados por cardiólogos
Hosting	Railway	—	Ejecuta el backend y la API de IA
Código fuente	Git + GitHub	—	Versiones y despliegue automático
Compilación Android	Gradle + Android SDK + JDK 17	—	Genera el APK

¿Por qué aparece Kotlin si la app es React Native?

React Native corre **sobre** Android, no lo reemplaza. La interfaz y la lógica están en TypeScript, pero el SDK del Polar H10 solo existe como librería nativa de Android. Para usarlo se escribe un **módulo nativo en Kotlin** que hace de puente. Es la forma estándar de integrar hardware en React Native (sección 3).

03 Integración del sensor Polar H10

Esta es la pieza central del proyecto: cómo una app de React Native lee el ECG de una banda Polar H10 en tiempo real.

3.1 El sensor y su SDK oficial

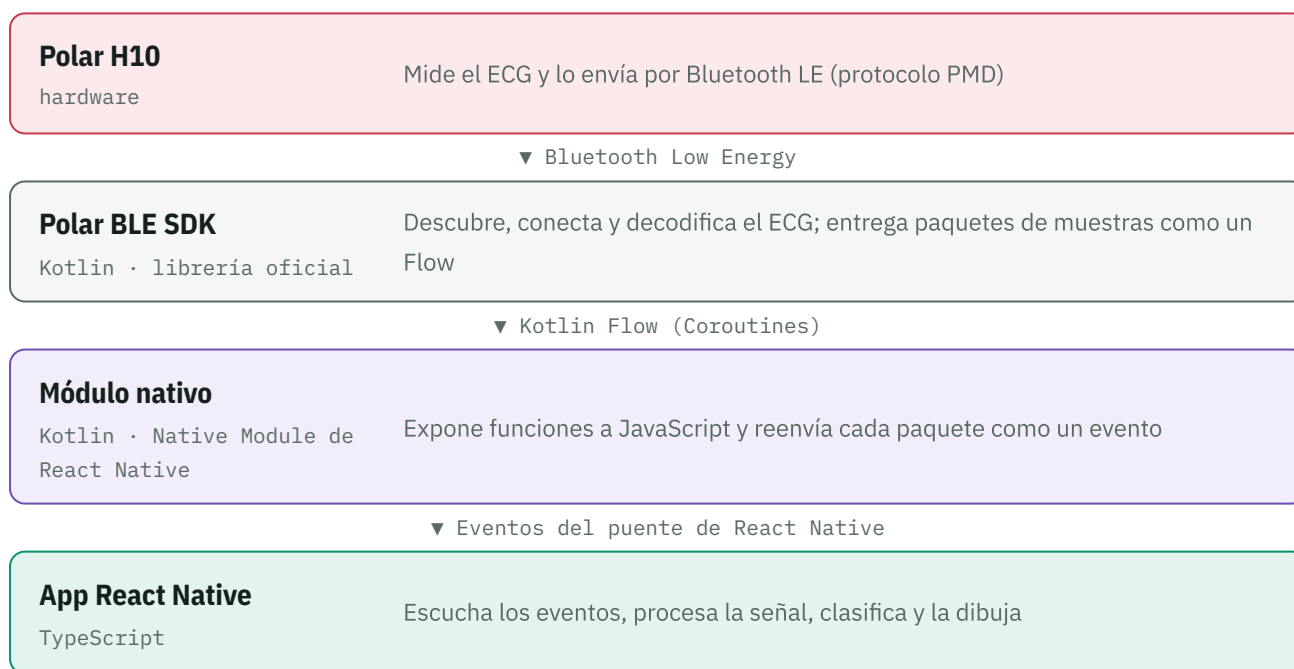
El **Polar H10** es una banda pectoral con electrodos que mide la actividad eléctrica del corazón en **una derivación**, a **130 muestras por segundo**, en **microvoltios (μV)**. Transmite por Bluetooth Low Energy usando un protocolo propio de Polar para datos en bruto, llamado PMD (*Polar Measurement Data*).

Ese protocolo no está disponible en los servicios Bluetooth estándar, así que para obtener el ECG en bruto se usa el **Polar BLE SDK**, la librería oficial y gratuita de Polar:

- **Repositorio oficial:** `github.com/polarofficial/polar-ble-sdk`
- **Distribución para Android:** JitPack, como `com.github.polarofficial:polar-ble-sdk`
- **Versión usada:** 8.3.0. Desde la versión 8, el SDK entrega los datos como *Flows* de Kotlin Coroutines y exige **Android 13 o superior** (API 33).

3.2 Arquitectura de la integración

La señal atraviesa cuatro capas desde el sensor hasta la pantalla:



3.3 Paso 1 · Agregar el SDK al proyecto Android

El SDK se descarga automáticamente al compilar. Se declara el repositorio JitPack y la dependencia:

android/build.gradle

```
allprojects {  
    repositories { maven { url "https://jitpack.io" } }  
}
```

android/app/build.gradle

```
dependencies {
  implementation("com.github.polarofficial:polar-ble-sdk:8.3.0")
  implementation("org.jetbrains.kotlinx:kotlinx-coroutines-android:1.10.2")
}
// y minSdkVersion = 33, que exige la versión 8 del SDK
```

3.4 Paso 2 • Permisos de Android

Android exige declarar el uso de Bluetooth en el manifiesto y pedir permiso al usuario en tiempo de ejecución.

PERMISO	PARA QUÉ
<code>BLUETOOTH_SCAN</code>	Buscar dispositivos cercanos (Android 12+). Se declara con <code>neverForLocation</code> : la búsqueda no se usa para ubicar al usuario
<code>BLUETOOTH_CONNECT</code>	Conectarse al sensor (Android 12+)
<code>ACCESS_FINE_LOCATION</code>	GPS de la ambulancia que se adjunta a la alerta
<code>bluetooth_le</code> (feature)	Indica que la app requiere Bluetooth LE

Al tocar "Buscar Polar H10", la app pide estos permisos con la API `PermissionsAndroid` de React Native. Si el usuario los rechaza, no se inicia la búsqueda.

3.5 Paso 3 • Funciones del SDK utilizadas

De todo lo que ofrece el Polar BLE SDK, CardioAlert usa estas funciones:

FUNCIÓN DEL SDK	QUÉ HACE	USO EN CARDIOALERT
<code>PolarBleApiDefaultImpl.defaultImplementation()</code>	Crea la instancia del SDK con las funciones que se activarán	Se activan: frecuencia cardíaca, streaming en línea, batería e información del dispositivo
<code>setApiCallback()</code>	Registra un receptor de eventos del sensor	Avisa cuando el sensor se conecta o desconecta y cuando informa su batería
<code>searchForDevice()</code>	Busca dispositivos Polar cercanos (Flow)	Lista de sensores en la pantalla "Conectar Sensor"
<code>connectToDevice(id)</code>	Conecta con un sensor por su ID	Al tocar un sensor de la lista (ej. <code>15F5CE35</code>)
<code>requestStreamSettings(id, ECG)</code>	Consulta las configuraciones de ECG que admite el sensor	Se elige la de mayor resolución (<code>maxSettings()</code>)
<code>startEcgStreaming(id, ajustes)</code>	Inicia el ECG en vivo (Flow de paquetes)	Cada muestra trae voltaje (μV) y marca de tiempo (ns)

FUNCIÓN DEL SDK	QUÉ HACE	USO EN CARDIOALERT
<code>startHrStreaming(id)</code>	Inicia frecuencia cardíaca e intervalos R-R	BPM e intervalo R-R que muestra la pantalla de monitoreo
<code>disconnectFromDevice(id)</code>	Cierra la conexión	Al terminar la sesión o desconectar

3.6 Paso 4 · El puente entre Kotlin y React Native

React Native permite crear **Native Modules**: clases nativas cuyas funciones se pueden llamar desde JavaScript. La integración usa estos componentes de React Native:

COMPONENTE DE REACT NATIVE	PAPEL EN LA INTEGRACIÓN
<code>ReactContextBaseJavaModule</code>	Clase base del módulo nativo; su nombre (<code>PolarModule</code>) es con el que lo encuentra JavaScript
<code>@ReactMethod</code> + <code>Promise</code>	Marca las funciones que JavaScript puede llamar (buscar, conectar, iniciar ECG...) y les devuelve éxito o error
<code>RCTDeviceEventEmitter</code>	Envía eventos de Kotlin a JavaScript: cada paquete de ECG, cambios de conexión, batería y errores
<code>Arguments</code> / <code>WritableMap</code>	Convierte los datos de Kotlin a objetos que JavaScript entiende
<code>ReactPackage</code>	Registra el módulo en la app (en <code>MainApplication.kt</code>)
<code>NativeModules</code> / <code>NativeEventEmitter</code>	Del lado de JavaScript: acceden al módulo y escuchan sus eventos

El patrón para el ECG es: el SDK entrega un *Flow*, una corrutina lo recorre y, por cada paquete, se emite un evento hacia JavaScript. En forma simplificada:

```
// Kotlin: se consulta la mejor configuración y se inicia el stream
val ajustes = api.requestStreamSettings(id, PolarDeviceDataType.ECG).maxSettings()
api.startEcgStreaming(id, ajustes).collect { paquete ->
    // cada muestra: voltaje en µV + marca de tiempo en ns
    enviarEventoAJavaScript("PolarEcgData", paquete.samples)
}
```

```
// TypeScript: la app escucha ese evento
const emisor = new NativeEventEmitter(NativeModules.PolarModule);
emisor.addListener('PolarEcgData', paquete => procesar(paquete.samples));
```

Eventos que el módulo envía a la app

EVENTO	CONTENIDO
<code>PolarDeviceFound</code>	ID, nombre y potencia de señal (RSSI) de cada sensor encontrado
<code>PolarConnectionState</code>	Conectando / conectado / desconectado

EVENTO	CONTENIDO
PolarEcgData	Paquete de muestras de ECG en μV con sus marcas de tiempo
PolarHrData	BPM, intervalos R-R en ms y contacto con la piel
PolarBattery	Nivel de batería del sensor (%)
PolarError	Errores de búsqueda, conexión o streaming, con su mensaje

3.7 Secuencia completa de conexión

- 1 Permisos.** El paramédico toca "Buscar Polar H10" y la app pide los permisos de Bluetooth.
- 2 Búsqueda.** Se llama a `searchForDevice()` ; cada sensor encontrado aparece en la lista con su ID y su señal.
- 3 Conexión.** Al tocar un sensor se llama a `connectToDevice(id)` . El callback del SDK confirma la conexión y la app muestra "Conectado ✓" y la batería.
- 4 Streaming.** Al iniciar el monitoreo se activan en paralelo el stream de ECG (130 Hz) y el de frecuencia cardíaca.
- 5 Recepción.** Cada paquete llega a la app como evento `PolarEcgData` y se dibuja en vivo.
- 6 Cierre.** Al finalizar la sesión se detienen los streams y se llama a `disconnectFromDevice(id)` .

Plataforma

El módulo nativo está hecho para **Android**. En iPhone la app no se cae: al no encontrar el módulo, usa el modo de señal simulada. La versión iOS requeriría portar el módulo a Swift con la versión iOS del mismo SDK.

04 Del sensor al diagnóstico

Una vez que el ECG llega a la app, pasa por un preprocesamiento que debe ser **idéntico** al que se usó para entrenar el modelo. Si difieren, el modelo recibe señales distintas de las que aprendió.

ETAPA	QUÉ HACE	POR QUÉ
Ventana	Guarda los últimos 10 s de ECG y genera una ventana nueva cada 2.5 s	El modelo analiza 10 s de ritmo; reclasificar cada 2.5 s da resultados fluidos
Remuestreo	Convierte de 130 Hz (Polar) a 250 Hz	El modelo se entrenó a 250 Hz
Filtro pasa-banda	Deja pasar de 0.5 a 40 Hz	Elimina la deriva de la línea base (respiración, movimiento) y el ruido muscular
Normalización	Z-score: media 0, desviación 1	El resultado no depende de la amplitud del equipo
Clasificación	Envía las 2500 muestras a la API de IA	Devuelve la clase y la probabilidad de cada una
Explicación	Oculto tramos de 50 ms y mide cuánto cae la confianza	Muestra qué partes del ECG pesaron en la decisión (XAI por perturbación)

Si la clase no es Normal y la confianza supera el **85%**, el celular vibra y la pantalla pasa a modo alerta. El paramédico decide si envía la alerta al hospital.

Modos de diagnóstico

Modelo IA

Cada ventana se clasifica con la red CNN-BiLSTM entrenada. Es el modo para pacientes reales. La etiqueta "MODELO IA" aparece junto al diagnóstico.

Simulación

Muestra el escenario elegido (Normal, AFib o Isquemia) para demostrar cada flujo sin un paciente que presente la condición. Siempre queda marcado como "SIMULADO".

05 Conexión con Supabase

Supabase reúne tres servicios en uno, a los que la app y el backend se conectan con la librería oficial `@supabase/supabase-js`.

SERVICIO	CÓMO SE CONECTA	USO
Auth	La app llama a <code>auth.signInWithPassword()</code> con la llave pública y recibe un token de sesión (JWT)	Login de paramédicos, médicos y administradores
PostgreSQL	Solo el backend escribe, con la llave secreta. La app nunca escribe datos clínicos directamente	Perfiles, sesiones, clasificaciones y alertas

SERVICIO	CÓMO SE CONECTA	USO
Realtime	La app se suscribe con <code>channel().on('postgres_changes', ...)</code> a la tabla de alertas	El médico se entera al instante de una alerta nueva

Tablas

TABLA	GUARDA
<code>profiles</code>	Nombre y rol de cada usuario (única fuente del rol)
<code>sessions</code>	Cada monitoreo: código anónimo del paciente, ID del sensor, inicio, fin y último diagnóstico
<code>classifications</code>	Cada cambio de diagnóstico con su confianza
<code>alerts</code>	Alertas enviadas: tipo, confianza, ECG, frecuencia cardíaca, R-R, GPS y estado

Seguridad de los datos

Las cuatro tablas tienen **Row Level Security**: cada paramédico ve solo sus sesiones y alertas, y el médico ve todas. Sin iniciar sesión no se ve ninguna fila. Los pacientes se identifican con un código anónimo (PAC-1234), nunca con su nombre.

06 Backend y API de inteligencia artificial

Backend (Node.js + Express)

Recibe las operaciones de la app y las guarda en Supabase. En cada petición verifica el token de sesión, lee el rol del usuario y valida los datos con Zod.

RUTA	QUIÉN	FUNCIÓN
POST /sessions	Paramédico	Inicia una sesión de monitoreo
POST /sessions/:id/classifications	Dueño de la sesión	Registra un diagnóstico
POST /sessions/:id/end	Dueño de la sesión	Cierra la sesión
GET /sessions	Todos	Historial según el rol
POST /alerts	Dueño de la sesión	Envía una alerta al hospital
GET /alerts	Todos	Alertas según el rol
POST /alerts/:id/status	Médico o admin	Aceptar o derivar una alerta
GET /users · POST /users	Administrador	Listar y crear cuentas de paramédico o médico

API de IA (Python + FastAPI + TensorFlow)

Carga el modelo entrenado (`.keras`) con TensorFlow al arrancar y lo expone por HTTPS, protegido con una clave (`X-API-Key`).

RUTA	RECIBE	DEVUELVE
GET /health	—	Si el modelo está cargado
POST /predict	2500 muestras (10 s a 250 Hz)	Clase, confianza y probabilidad de cada clase
POST /explain	2500 muestras	Importancia de cada tramo de 50 ms (XAI)

El modelo

Red **CNN-BiLSTM**: las capas convolucionales reconocen la **forma** de cada latido (complejo QRS, segmento ST) y la capa LSTM bidireccional analiza la **secuencia** de latidos, clave para detectar el ritmo irregular de la fibrilación auricular. Se entrena en Google Colab con datos públicos de PhysioNet (MIT-BIH Arrhythmia, MIT-BIH Atrial Fibrillation y European ST-T), dividiendo los datos **por paciente**, de modo que la evaluación usa pacientes que el modelo nunca vio.

07 Configuración

La app toma su configuración de un archivo `.env` que se incorpora al APK al compilar. Todo cambio en ese archivo requiere generar el APK de nuevo.

VARIABLE (APP)	CONTENIDO
<code>SUPABASE_URL</code>	Dirección del proyecto de Supabase
<code>SUPABASE_ANON_KEY</code>	Llave pública de Supabase (segura en la app gracias a RLS)
<code>API_BASE_URL</code>	Dirección del backend en Railway
<code>ML_API_URL</code> · <code>ML_API_KEY</code>	Dirección y clave de la API de IA
<code>ALERT_CONFIDENCE_THRESHOLD</code>	Confianza mínima para disparar una alerta (0.85)

VARIABLE (SERVIDORES, EN RAILWAY)	CONTENIDO
<code>SUPABASE_URL</code> · <code>SUPABASE_SERVICE_ROLE_KEY</code>	Backend: dirección y llave secreta de Supabase
<code>ML_API_KEY</code>	API de IA: la clave que deben enviar las peticiones

Importante

La llave secreta de Supabase solo debe estar en Railway. Nunca en la app, en el repositorio ni en documentos compartidos: con ella se salta toda la seguridad de la base de datos.

08 Compilación del APK

El APK se genera con la cadena de compilación estándar de Android, por línea de comandos. No hace falta Android Studio.

HERRAMIENTA	FUNCIÓN
Node.js y npm	Instalan las dependencias de JavaScript de la app
JDK 17 (OpenJDK)	Necesario para compilar Kotlin y ejecutar Gradle
Android SDK (command-line tools)	Plataforma Android, herramientas de compilación y NDK
Gradle (<code>./gradlew</code>)	Sistema de compilación de Android: une el código nativo, las librerías y el JavaScript
Metro + Hermes	Empaquetan y optimizan el código TypeScript dentro del APK

Pasos

- 1 Instalar las dependencias de la app, desde `CardioAlertMobile/` :

```
npm install
```

- 2 Completar el archivo `CardioAlertMobile/.env` (sección 7).

- 3 Compilar, desde `CardioAlertMobile/android/` :

```
./gradlew assembleRelease -PreactNativeArchitectures=arm64-v8a
```

`assembleRelease` genera la versión final; `arm64-v8a` la compila solo para los procesadores de los celulares Android actuales, lo que reduce el tamaño a unos 36 MB.

- 4 El APK queda en `android/app/build/outputs/apk/release/app-release.apk` .

Instalación en un celular

- 1 Enviar el archivo `.apk` al celular (WhatsApp, Drive o cable).
- 2 Abrirlo y permitir "instalar apps de origen desconocido" si Android lo pide.
- 3 Si aparece Google Play Protect, elegir "Instalar de todas formas": aparece porque la app no viene de la Play Store.

Requisitos y alcance

- El celular debe tener **Android 13 o superior**, por exigencia del Polar SDK 8.
- El APK está firmado con la llave de depuración de React Native: sirve para instalarlo y hacer demostraciones, pero para publicarlo en Google Play habría que firmarlo con una llave propia.

09 Despliegue en la nube

El código está en un solo repositorio de GitHub con cuatro carpetas. Railway despliega automáticamente dos servicios a partir de él.

CARPETA	CONTENIDO	SE DESPLIEGA EN
CardioAlertMobile/	App móvil	APK (no va a la nube)
backend/	API Express	Railway · servicio "backend"
ml-api/	API de IA y el modelo <code>.keras</code>	Railway · servicio "ml-api"
supabase/	<code>schema.sql</code> : tablas y reglas de seguridad	Se ejecuta una vez en Supabase

- 1 **Supabase:** crear el proyecto y ejecutar `supabase/schema.sql` en el SQL Editor.
- 2 **Railway:** crear un servicio por carpeta desde el repositorio de GitHub, con *Root Directory* `/backend` o `/ml-api`, *Watch Paths* de su carpeta y sus variables (sección 7).
- 3 **Actualizar:** cada `git push` que cambia una carpeta redespiega solo ese servicio.
- 4 **Nuevo modelo:** reemplazar el archivo en `ml-api/model/` y hacer push.

10 Guía de uso por rol

Administrador

- 1 Iniciar sesión eligiendo el rol **Admin**. La app abre la pantalla de Administración.
- 2 En "Crear usuario", completar nombre, correo, contraseña inicial (mínimo 8 caracteres) y elegir **Paramédico** o **Médico**.
- 3 Tocar **Crear usuario**. La cuenta queda activa de inmediato y aparece en la lista. Se le comunica la contraseña al usuario por un canal privado.

Las cuentas de administrador no se crean desde la app: se crean en el panel de Supabase, por seguridad.

Paramédico

- 1 Humedecer los electrodos de la banda Polar H10 y colocarla en el pecho del paciente. El sensor solo se enciende con contacto con la piel.
- 2 Iniciar sesión con el rol **Paramédico**. En Inicio, tocar la tarjeta del sensor y luego **Buscar Polar H10**; aceptar los permisos.
- 3 Tocar el sensor en la lista. Cuando diga **Conectado ✓**, tocar **Iniciar Monitoreo**.
- 4 A los 10 s aparece el primer diagnóstico con su confianza. **Ver Explicación IA** muestra qué zonas del ECG pesaron en la decisión.
- 5 Si hay una anomalía, **Enviar Alerta** abre la confirmación con el resumen y el GPS; **Enviar Alerta al Hospital** la envía.
- 6 **Finalizar sesión** cierra el monitoreo y lo guarda en el historial.

Médico

- 1 Iniciar sesión con el rol **Médico**. La app abre el **Monitor Hospital**.
- 2 Las alertas nuevas aparecen al instante, con un borde que late mientras están pendientes. Al tocarlas se ven el ECG, la confianza, la frecuencia cardíaca, el R-R y la ubicación.
- 3 Tocar **Aceptar** o **Derivar**. Una alerta solo puede responderse una vez.

11 Solución de problemas

SÍNTOMA	CAUSA PROBABLE	SOLUCIÓN
El Polar no aparece en la búsqueda	La banda no toca la piel o los electrodos están secos	Humedecer los electrodos y ajustar la banda al pecho
El Polar no aparece aunque está puesto	Bluetooth o ubicación apagados, o la app Polar Flow/Beat está conectada a la banda	Activar Bluetooth y ubicación, y cerrar las apps de Polar
"Permisos de Bluetooth denegados"	Se rechazaron los permisos	Ajustes de Android → Apps → CardioAlert → Permisos → Dispositivos cercanos
El APK no se instala	Android anterior a la versión 13	Usar un celular con Android 13 o superior
"Esta cuenta no tiene el rol..."	Se eligió un rol distinto del asignado a la cuenta	Elegir el rol correcto en el login
Error de la API de inferencia	Sin internet o la API se está reiniciando	Verificar la conexión; mientras tanto se puede usar el modo Simulación
La alerta no se envía	El celular del paramédico no tiene internet	La app muestra el error; reintentar el envío cuando vuelva la conexión
El médico no ve una alerta enviada	El celular del médico no tiene internet	Al recuperar la conexión, el Monitor Hospital vuelve a cargar las alertas

12 Referencias

- **Polar BLE SDK** (repositorio y documentación oficial): github.com/polarofficial/polar-ble-sdk
- **React Native: Native Modules para Android**: reactnative.dev/docs/native-modules-android
- **Supabase**: autenticación, Row Level Security y Realtime: supabase.com/docs
- **PhysioNet**: MIT-BIH Arrhythmia Database, MIT-BIH Atrial Fibrillation Database y European ST-T Database: physionet.org
- **TensorFlow / Keras**: tensorflow.org · **FastAPI**: fastapi.tiangolo.com · **Express**: expressjs.com
- **Railway**: despliegue de servicios desde GitHub: docs.railway.com
- Paralič, J. et al. (2023). *Perturbation-Based Explainable AI for ECG Sensor Data*. Applied Sciences.